Dataflow Analysis (1)

CSE552 Program Analysis — Lecture 7

Jaemin Hong

Dataflow Analysis Overview

- Starts with a CFG and a complete lattice (with a finite height)
  - A lattice element represents abstract information for a CFG node
  - The lattice may be
    - fixed for all programs, or
    - parameterized by the program
- To each node v, we assign a constraint variable $[\![v]\!]$ ranging over the lattice elements

Dataflow Constraints and Fixed Points

- For each node, we define a dataflow constraint that relates the variable to those of other nodes
- If all constraints are (in-) equations with monotone right-hand sides, we can use the fixed-point algorithm

Monotone Framework

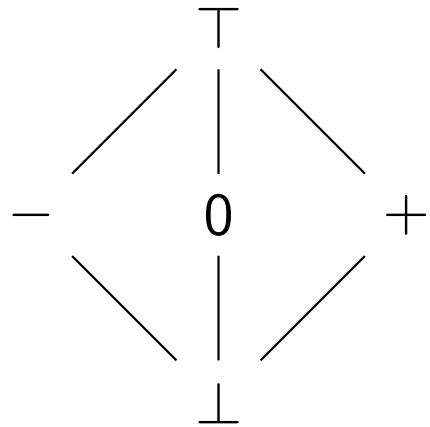
- The combination of a complete lattice and a space of monotone functions
- Can be instantiated by specifying the CFG and the rules for assigning dataflow constraints

Syntax

$$
\begin{array}{rcl}
\text{Node } v & ::= & x\texttt{=}e \mid \texttt{if } x \mid \texttt{entry} \mid \texttt{return} \\
\text{Expression } e & ::= & x \mid n \mid \texttt{input()} \mid e \; op \; e
\end{array}
$$

(No functions, pointers, and compound types for now)

Sign Analysis — Lattice



- $[\![v]\!] \in \text{State} = \text{Var} \to \text{Sign}$
- State^n expresses information for all CFG nodes, where n is the number of nodes

Sign Analysis — JOIN

- JOIN(v) combines the abstract states from the predecessors of v:

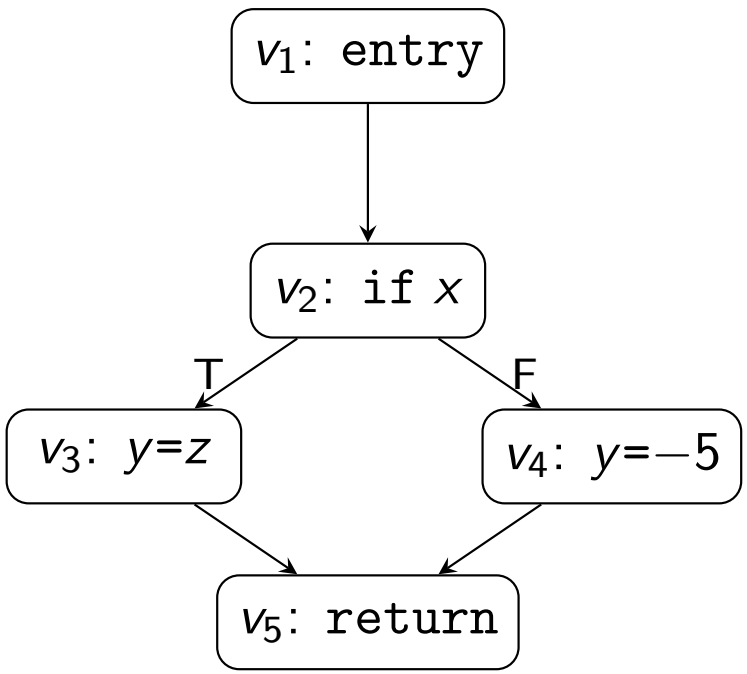
$$\text{JOIN}(v) = \bigsqcup_{u \in \text{pred}(v)} [\![u]\!]$$

- Precisely speaking, JOIN(v) is a function

$$\text{JOIN}(v) : \text{State}^n \to \text{State}$$

$$\text{JOIN}(v)([\![v_1]\!], \ldots, [\![v_n]\!]) = [\![v_i]\!] \sqcup [\![v_j]\!] \sqcup \cdots$$

Sign Analysis — JOIN Example



$$\text{JOIN}(v_1) = \bot$$
$$\text{JOIN}(v_2) = [\![v_1]\!]$$
$$\text{JOIN}(v_3) = [\![v_2]\!]$$
$$\text{JOIN}(v_4) = [\![v_2]\!]$$
$$\text{JOIN}(v_5) = [\![v_3]\!] \sqcup [\![v_4]\!]$$

Sign Analysis — Constraint Rules

- x=e: $[\![v]\!] = \text{JOIN}(v)[x \mapsto \text{eval}(\text{JOIN}(v), e)]$

$\text{eval} : (\text{Var} \to \text{Sign}) \times \text{Expression} \to \text{Sign}$

- $\text{eval}(\sigma, x) = \sigma(x)$
- $\text{eval}(\sigma, n) = \text{sign}(n)$
- $\text{eval}(\sigma, \texttt{input()}) = \top$
- $\text{eval}(\sigma, e_1 \ op \ e_2) = \widehat{op}(\text{eval}(\sigma, e_1), \text{eval}(\sigma, e_2))$

Sign Analysis — Abstract Addition

$\widehat{+}$	$\bot$	$-$	0	$+$	$\top$
$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$
$-$	$\bot$	$-$	$-$	$\top$	$\top$
0	$\bot$	$-$	0	$+$	$\top$
$+$	$\bot$	$\top$	$+$	$+$	$\top$
$\top$	$\bot$	$\top$	$\top$	$\top$	$\top$

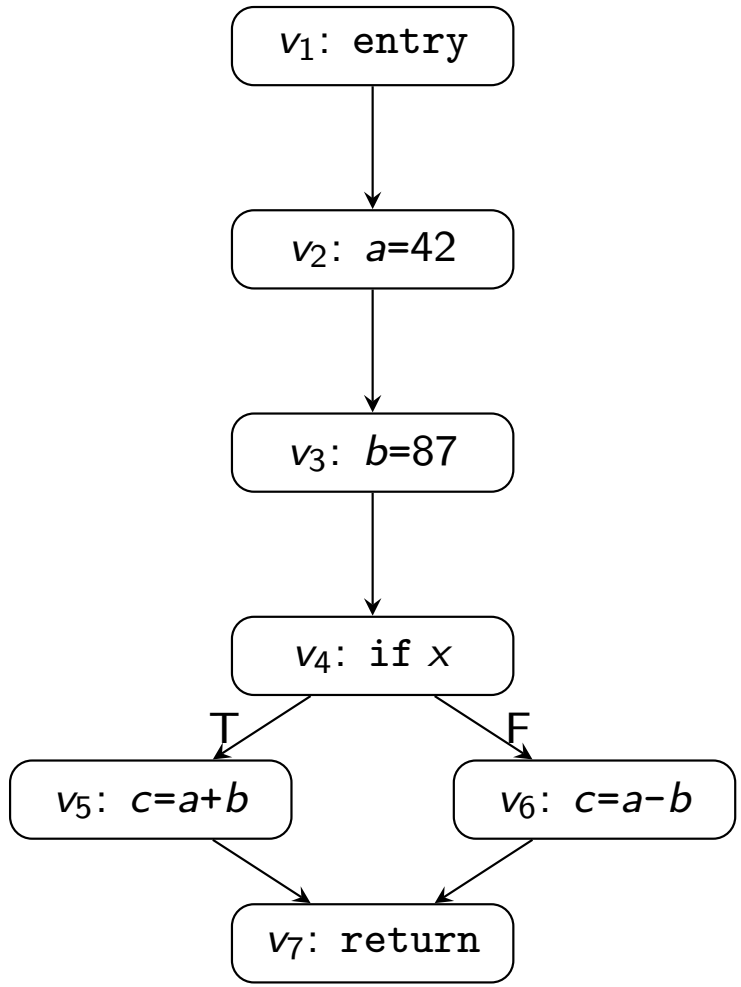
Sign Analysis — Remaining Rules

- `entry`: $[\![v]\!] = \top$
- Others: $[\![v]\!] = \text{JOIN}(v)$

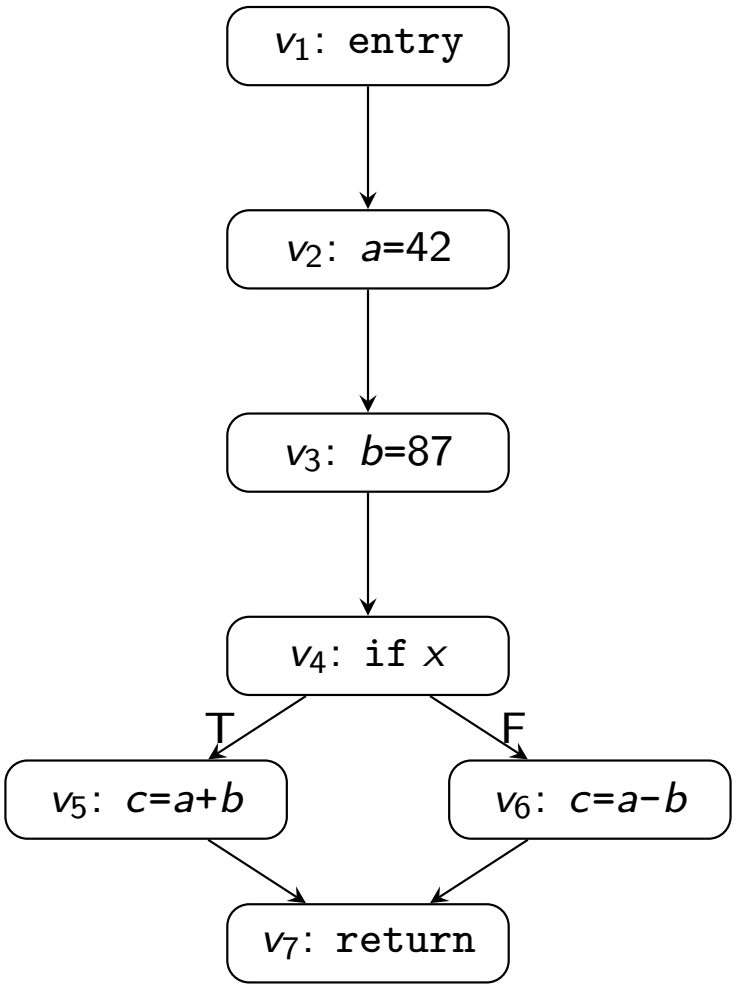
- While $\sqcap$ exists, we only use $\sqcup$
  - This is common

Sign Analysis — Constraint Example

```
a = 42;
b = 87;
if x {
   c = a + b;
} else {
   c = a - b;
}
return;
```



Sign Analysis — Constraint Example (Constraints)



$$[\![v_1]\!] = \top$$
$$[\![v_2]\!] = [\![v_1]\!][a \mapsto +]$$
$$[\![v_3]\!] = [\![v_2]\!][b \mapsto +]$$
$$[\![v_4]\!] = [\![v_3]\!]$$
$$[\![v_5]\!] = [\![v_4]\!][c \mapsto \widehat{+}([\![v_4]\!](a), [\![v_4]\!](b))]$$
$$[\![v_6]\!] = [\![v_4]\!][c \mapsto \widehat{-}([\![v_4]\!](a), [\![v_4]\!](b))]$$
$$[\![v_7]\!] = [\![v_5]\!] \sqcup [\![v_6]\!]$$

Monotonicity

- Function composition preserves monotonicity
- $\sqcup$ is monotone
- Map update is monotone
- $\widehat{op}$ is monotone
- $\mathsf{eval}(\_, e) : (\mathsf{Var} \to \mathsf{Sign}) \to \mathsf{Sign}$ is monotone for every e

Monotonicity of Addition

$\widehat{+}$	$\bot$	$-$	0	$+$	$\top$
$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$
$-$	$\bot$	$-$	$-$	$\top$	$\top$
0	$\bot$	$-$	0	$+$	$\top$
$+$	$\bot$	$\top$	$+$	$+$	$\top$
$\top$	$\bot$	$\top$	$\top$	$\top$	$\top$

Solving Constraints

- $f : \text{State}^n \to \text{State}^n$
- $f([\![v_1]\!], \ldots, [\![v_n]\!]) = (f_1([\![v_1]\!]), \ldots, f_n([\![v_n]\!]))$

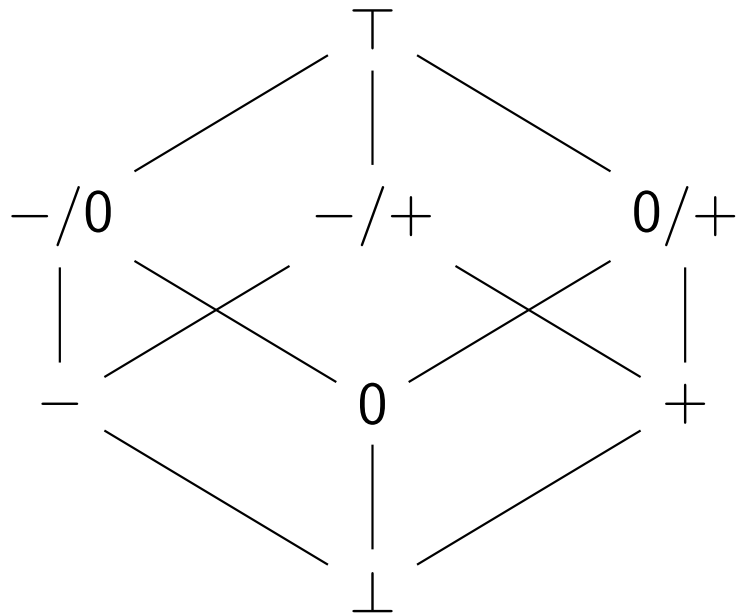
We can compute $\text{lfp}(f)$ using:

NaiveFixedPointAlgorithm(f)**:**

$x \leftarrow \bot$
while $x \neq f(x)$ **do**
  $x \leftarrow f(x)$
return x

Precision — Refined Sign Lattice

- Adding abstract values can improve precision
  - e.g., $-/0$, $-/+$, $0/+$

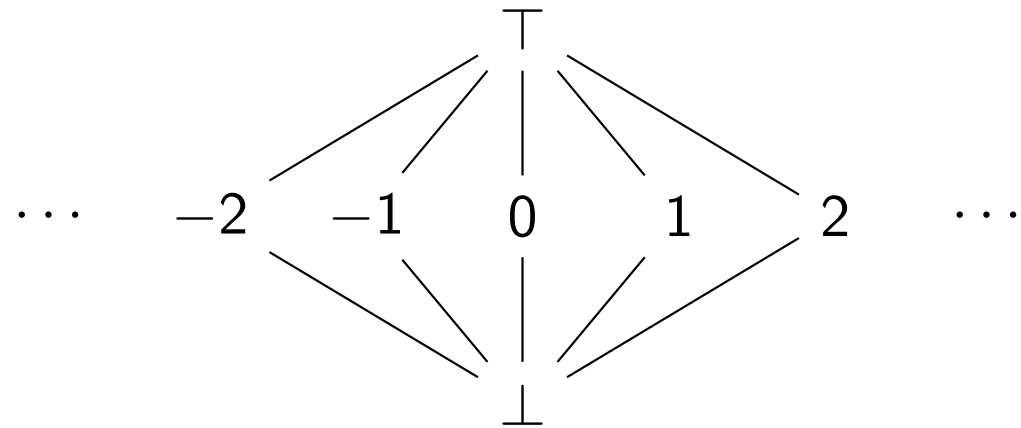


Applications of Sign Analysis

- In theory, e.g., can detect division-by-zero errors
  - Identify division whose divisor is 0 or $\top$
  - Would have too many false alarms
- More powerful analysis techniques can be useful
  - Interval domain
  - Path sensitivity

Constant Propagation — Lattice

$$\mathsf{State} = \mathsf{Var} \to \mathsf{flat}(\mathbb{Z})$$



Constant Propagation — Constraint Rules

- x=e: $[\![v]\!] = \text{JOIN}(v)[x \mapsto \text{eval}(\text{JOIN}(v), e)]$
- `entry`: $[\![v]\!] = \top$
- Others: $[\![v]\!] = \text{JOIN}(v)$

$\text{eval} : (\text{Var} \to \text{Flat}(\mathbb{Z})) \times \text{Expression} \to \text{Flat}(\mathbb{Z})$

- $\text{eval}(\sigma, x) = \sigma(x)$
- $\text{eval}(\sigma, n) = n$
- $\text{eval}(\sigma, \texttt{input()}) = \top$
- $\text{eval}(\sigma, e_1 \ op \ e_2) = \widehat{op}(\text{eval}(\sigma, e_1), \text{eval}(\sigma, e_2))$

Constant Propagation — Abstract Operator

$$a \mathbin{\widehat{op}} b = \begin{cases} \bot & \text{if } a = \bot \text{ or } b = \bot \\ \top & \text{otherwise, if } a = \top \text{ or } b = \top \\ a \mathbin{op} b & \text{otherwise} \end{cases}$$

Constant Propagation — Application

Compiler optimization:

Before:

```
a = 3;
b = a * 2;
c = a + input();
a = a * b;
e = a + c;
```

After:

```
a = 3;
b = 6;
c = 3 + input();
a = 18;
e = 18 + c;
```

Fixed-Point Algorithm — Motivation

- We need to find $\mathrm{lfp}(f)$ where $f : \mathrm{State}^n \to \mathrm{State}^n$
- NaiveFixedPointAlgorithm computes every f_i in each iteration
  - Much of the computation is redundant

Example: $x = (x_1, \ldots, x_7)$

$$f_1(x) = \top \qquad f_5(x) = x_4[c \mapsto \widehat{+}(x_4(a), x_4(b))]$$
$$f_2(x) = x_1[a \mapsto +] \qquad f_6(x) = x_4[c \mapsto \widehat{-}(x_4(a), x_4(b))]$$
$$f_3(x) = x_2[b \mapsto +] \qquad f_7(x) = x_5 \sqcup x_6$$
$$f_4(x) = x_3$$

Fixed-Point Algorithm — Exploiting Structure

Example: $x = (x_1, \ldots, x_7)$

$$
\begin{aligned}
f_1(x) &= \top & f_5(x) &= x_4[c \mapsto \widehat{+}(x_4(a), x_4(b))] \\
f_2(x) &= x_1[a \mapsto +] & f_6(x) &= x_4[c \mapsto \widehat{-}(x_4(a), x_4(b))] \\
f_3(x) &= x_2[b \mapsto +] & f_7(x) &= x_5 \sqcup x_6 \\
f_4(x) &= x_3
\end{aligned}
$$

- e.g., f_2 depends only on x_1, but the value of x_1 does not change in most iterations
- We can exploit the fact that our lattice is L^n and f consists of $f_1, \ldots, f_n$

Round Robin Algorithm

- $x = (x_1, \ldots, x_n), \quad f(x) = (f_1(x), \ldots, f_n(x))$

RoundRobin$(f_1, \ldots, f_n)$**:**

```
x ← ⊥
while x ≠ f(x) do
    for i in 1 . . . n :
        x_i ← f_i(x)
return x
```

- One iteration of the while loop does not give the same result as one iteration of NaiveFixedPointAlgorithm in general
- However, always terminates and produces lfp(f)
- The number of iterations until the fixed point may be smaller

Round Robin — Observations

RoundRobin$(f_1, \ldots, f_n)$**:**

$x \leftarrow \bot$

while $x \neq f(x)$ **do**

for i *in* $1 \ldots n$ **:**

$x_i \leftarrow f_i(x)$

return x

- The order of the iterations $i := 1 \ldots n$ is irrelevant with the final result
- We need to update x_i if $x_i \neq f_i(x)$ to reach the fixed point
- We do not need to update x_i if $x_i = f_i(x)$

Chaotic Iteration

ChaoticIteration$(f_1, \ldots, f_n)$**:**

$x \leftarrow \bot$

while $x \neq f(x)$ **do**

choose $i \in \{1, \ldots, n\}$ s.t. $x_i \neq f_i(x)$

$x_i \leftarrow f_i(x)$

return x

- Always terminates and produces $\mathrm{lfp}(f)$
- The number of assignments until the fixed point may be smaller

Chaotic Iteration — Problems

ChaoticIteration$(f_1, \ldots, f_n)$**:**

$x \leftarrow \bot$

while $x \neq f(x)$ **do**

choose $i \in \{1, \ldots, n\}$ s.t. $x_i \neq f_i(x)$

$x_i \leftarrow f_i(x)$

return x

- Not practical, as efficiency depends on the choice of i
- Finding i requires computing f_i's so it is expensive

Worklist Algorithm — Observation

- f_i typically uses only a few of $x_1, \ldots, x_n$
- We can record the nodes that need recomputation based on what we updated, rather than newly finding them every time

Example:

$$
\begin{aligned}
f_1(x) &= \top & f_5(x) &= x_4[c \mapsto \widehat{+}(x_4(a), x_4(b))] \\
f_2(x) &= x_1[a \mapsto +] & f_6(x) &= x_4[c \mapsto \widehat{-}(x_4(a), x_4(b))] \\
f_3(x) &= x_2[b \mapsto +] & f_7(x) &= x_5 \sqcup x_6 \\
f_4(x) &= x_3
\end{aligned}
$$

Worklist Algorithm — dep

- We introduce a map to keep this information
  - dep : Node $\to \mathcal{P}$(Node)
  - dep(v) = the set of nodes whose information depends on the information of v
- For the sign analysis and constant propagation analysis, dep = succ
- When the information of v is updated, only the nodes in dep(v) need to be recomputed

Worklist Algorithm — dep Example

$$[\![v_5]\!] = [\![v_4]\!][c \mapsto \widehat{+}([\![v_4]\!](a), [\![v_4]\!](b))]$$
$$[\![v_6]\!] = [\![v_4]\!][c \mapsto \widehat{-}([\![v_4]\!](a), [\![v_4]\!](b))]$$

$$\mathsf{dep}(v_4) = \{v_5, v_6\}$$

Worklist Algorithm — Pseudocode

SimpleWorkListAlgorithm$(f_1, \ldots, f_n)$**:**

$x \leftarrow \bot$
$W \leftarrow \{v_1, \ldots, v_n\}$
while $W \neq \emptyset$ **do**
  $v_i \leftarrow W$.removeOne()
  $y \leftarrow f_i(x)$
  if $y \neq x_i$ **:**
    $x_i \leftarrow y$
    $W \leftarrow W \cup \mathsf{dep}(v_i)$
return x

- W is called the worklist
- Always terminates and produces $\mathsf{lfp}(f)$

Worklist Algorithm — Time Complexity

If $|\text{dep}(v)|$ is bounded by a constant for all nodes v, the worst-case time complexity is

$$O(n \cdot h \cdot k)$$

where

- n is the number of CFG nodes
- h is the height of the lattice $L = \text{State}$
- k is the worst-case time required to compute f_i

Worklist Algorithm — Potential Improvements

- Handle strongly connected components (cycles) separately
- Use a priority queue for the worklist
- Make the dependence information more precise by allowing dep to consider $x_1, \ldots, x_n$ in addition to v

Summary

- Dataflow analysis assigns constraint variables over a lattice to CFG nodes and solves monotone constraints via fixed-point computation
- Sign analysis tracks the sign of variables; constant propagation tracks exact integer values
- The naive fixed-point algorithm recomputes all nodes each iteration; Round Robin and Chaotic Iteration improve on this
- The worklist algorithm uses dependency information (dep) to recompute only affected nodes, achieving $O(n \cdot h \cdot k)$ complexity